

TP 1

Objectifs :

- **Manipuler les branches en git**

Exercice 1 : Manipulation des branches localement

- Créer et initialiser, localement, un projet git ;
- Ajouter un fichier index.html (affichant un paragraphe lorem) ;
- Via les commandes git, ajouter et valider ce fichier au répertoire local. (git add ..., git commit ...)

Branche fn/tab :

- Créer et pointer sur une branche appelée fn/tab ;
- Ajouter au fichier index.html un titre, modifier le paragraphe et ajouter un tableau en bas.
- Ajouter et valider les modifications avec git ;

Branche fn/footer :

- Repointer sur la branche master ;
- Créer et pointer sur une nouvelle branche appelée fn/footer ;
- Ajouter un footer à notre fichier index.html (ex. : `<span>copyright 2022</span>`)
- Ajouter et valider les modifications avec git ;

Fusionner les deux branches fn/tab et fn/footer :

- Sur la branche fn/tab, lancer la commande pour la fusionner avec fn/footer : git merge fn/footer ;
- Vous pouvez voir les conflits à l'aide de git status ;
- Résolvez les conflits à l'aide de visual studio code en gardant toutes les lignes ajoutées à partir des deux branches ;
- Ajouter et valider les modifications avec git ;

Fusionner avec la branche master :

- Repointer sur la branche master ;
- Lancer la commande pour la fusionner avec fn/tab ;
- Ajouter et valider les modifications avec git ;

TP 1.1

Objectifs :

- **Manipulation de dépôts avec Git/GitLab**

Git / GitLab :

1. Créer un nouveau projet gitlab ;
2. A l'aide de git local, lancez une commande « *clone* » pour cloner votre répertoire distant dans un répertoire local de votre choix ;
3. Ajouter un fichier default.html et y insérer un contenu html ;
4. Ajouter et valider les modifications avec git ;
5. Apporter les modifications du fichier default.html au répertoire central à l'aide de la commande *push* ;
6. Modifier le fichier en GitLab ;
7. Télécharger les modifications en local à l'aide de la commande *pull* ;

Manipulation des branches dans un répertoire distant :

8. Localement, créer une nouvelle branche appelée *design* ;
9. Ajouter un fichier *style.css* et lier le au fichier default.html ;
10. Ajouter et valider les modifications avec git (*git add . / git commit -m 'message'*) ;
11. Apporter les modifications au répertoire distant à l'aide de push :

- Si on veut fusionner les modifications avec main, on écrit :

git push origin design

- Si on veut créer une branche dans le répertoire distant similaire à notre branche, on écrit :

git push -u origin design

Dans les deux cas, on peut faire une demande de fusion « merge request » en accédant à l'url proposé via le navigateur.

12. Sur Gitlab, vous pouvez accepter la requête de fusion ou la refuser.

TP 2

Objectifs :

- Manipulation de dépôts avec Git/GitLab

TP réalisé par binôme :

On souhaite créer une application « todo » : une application web permettant de gérer les tâches personnelles à faire, comme le montre la capture suivante :

ToDoList	
Task Title	Add
Arroser les plantes	Undo X
Terminer l'activité 7 du module Approche agile	Done X
Appeler le technicien qui répare mon ancien tél	Done X
Acheter le riz et du lait	Undo X

Cette application est en php/Mysql et elle doit être réalisée en binôme, en respectant les consignes suivantes :

- Le projet doit être géré et enregistré dans GitLab ou GitHub.
- L'un des stagiaires sera chargé par la réalisation de la partie front end et l'autre développera la partie back end.

Git / GitLab :

1. Créer un nouveau projet gitlab ;
2. A l'aide de git local, lancez une commande « **clone** » pour cloner le répertoire distant dans un répertoire local de votre choix ;
3. Ajouter un fichier index.php :

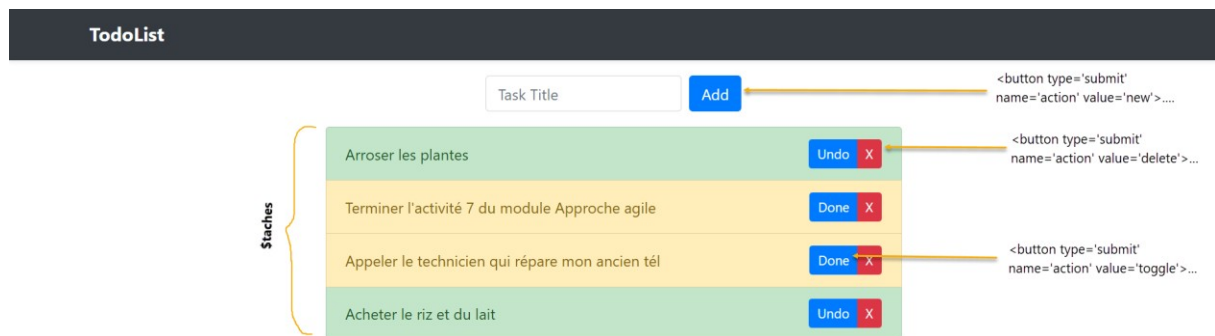
En haut de ce fichier, ajouter un ensemble de constantes définissant la configuration de la base de données :

```
// Database connection parameters.  
define('DB_USER', 'root');  
define('DB_PASS', '');
```

```
define('DB_NAME', 'todolist');
define('DB_HOST', '127.0.0.1');
define('DB_PORT', '3306');
```

4. Insérer par la suite, dans le même document index, la structure html ;
5. Ajouter et valider les modifications avec git ;
6. Apporter les modifications du fichier index.php au répertoire central à l'aide de la commande *push* ;

Réalisation du projet :



▪ Le développeur back-end:

- a. Il est amené à lire la liste des tâches enregistrées dans la table « todo » et les enregistrer sous forme d'une liste des objets dans une variable appelée « \$taches ». *(Cette liste doit être triée par date de création du plus récent au plus ancien)*
- b. Selon l'action reçue par la page index (une variable reçue en post), le développeur back end doit écrire les requêtes correspondantes, tel que :
 - **new** : sert à ajouter une nouvelle tâche à la table todo ;
 - **delete** : sert à supprimer la tâche de la table todo ;
 - **toggle** : sert à basculer la valeur du champ « done » entre true et false de la tâche en question.

Indication : pour faire ceci, vous pouvez utiliser :
update todo set done=1-done where id=...

A noter :

- Tous les formulaires envoient leurs valeurs avec la méthode « post »
- La tâche à traiter a toujours la valeur « id » comme name.
- Le code de cette partie doit être écrit en haut de la page index.

▪ Le développeur front-end:

En utilisant Bootstrap :

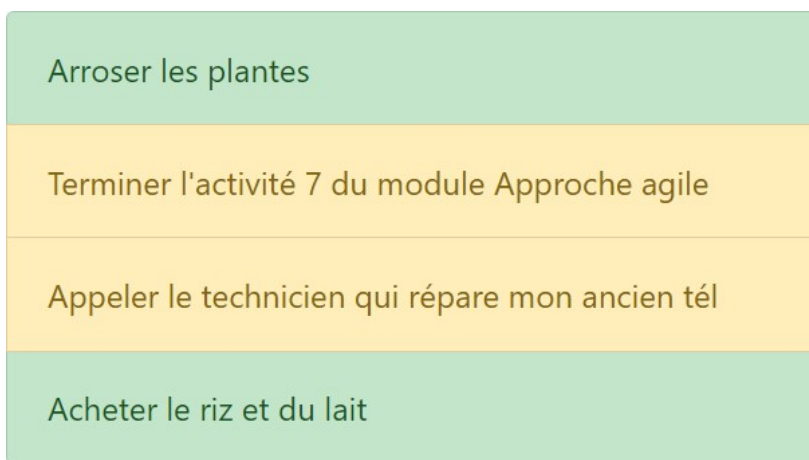
- a. Ajouter un navbar à la page index.php ;

- b. Ajouter un formulaire d'ajout d'une nouvelle tâche, tel que le « name » de la zone de texte est 'title' :

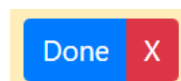


- c. Ajouter en bas, une liste des tâches, en se basant sur la variable *\$taches* ; cette variable sera remplie par la liste provenant de la table todo et c'est le développeur back-end qui s'en charge.

- Afficher les tâches effectuées en utilisant cette classe : `list-group-item-success`
- Et les tâches qui ne sont pas encore réalisées en : `list-group-item-warning`



- d. Ajouter pour chaque tâche, un formulaire se composant de :
- Un input de type hidden comprenant l'id de la tâche courante.
 - Deux boutons de type submit ; les deux auront le même name « action » et auront des valeurs différentes : toggle et delete.



Remarques :

- Chaque développeur doit créer sa branche et y travailler.
- A la fin de chaque question, testez vos travaux et lancez les commandes :
 - ✓ **git fetch** : pour télécharger les dernières modifications
 - ✓ **git diff origin/main** ou pointez sur la branche distante avec **git checkout origin/main** pour examiner les différences.
 - ✓ **git merge origin/main** : pour appliquer les changements au répertoire de travail.

✓ **git push -u origin nom_branche** : pour déposer les modifications dans le répertoire distant.

- L'un doit envoyer et traiter les merges request de l'autre.

Script de la base de données :

```
CREATE DATABASE IF NOT EXISTS todolist;
USE todolist;
CREATE TABLE IF NOT EXISTS `todo` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT,
  `title` varchar(2048) NOT NULL,
  `done` tinyint(1) NOT NULL DEFAULT '0',
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`)
);
```

TP 3

Objectifs :

- **Contribuer à un projet open source**

Le projet suivant est un projet public, chaque stagiaire doit y contribuer en proposant son code javascript :

https://gitlab.com/devowfs_adarissa/tp_calculatrice

- Le répertoire contient des ressources html/css/js permettant de créer une calculatrice web.
- Les codes html et css sont fournis quant au code javascript, il ne l'est pas.
- Le travail demandé est d'ajouter le code javascript au fichier code.js permettant de faire fonctionner les boutons de la calculatrice.

1. Créez votre fork du projet ;
2. Modifiez le fichier code.js, testez, validez (commit) et poussez (push) le vers votre fork ;
3. Répétez le 2. Jusqu'à l'obtention d'une version stable ;
4. Créez un merge request entre votre fork et le projet original.

TP 4

Objectifs :

- Analyser un code avec sonar-scanner
- Interpréter les résultats affichés par Sonarqube
- Utiliser sonarLint

Soit le projet « Activité 9_code source » ; Il s'agit du code source d'un projet d'un carnet d'adresse web écrit en php/mysql/bootstrap ;

- Testez l'application, et ajoutez le code nécessaire pour qu'elle soit fonctionnelle ;
- Scannez le projet avec sonar-scanner et examinez les résultats en sonarqube ;
- Corrigez, au niveau du code, le maximum possible des problèmes détectés ;
(Installez l'extension sonarLint sous visual studio code pour avoir des propositions d'améliorations en codant) ;
- Rescanner/corrigez jusqu'à obtenir le meilleur score de qualité possible.

TP 5

Objectifs :

- Rappel des concepts de la POO
- Initiation aux tests unitaires

1. Créer et initialiser un projet à l'aide de composer ; Ajouter phpunit comme dépendance de développement.

Soit la classe **Produit**, définit par les attributs suivants :

- **reference** : la chaîne unique permettant l'identification d'un produit
- **libelle** : le nom du produit
- **prix**
- **type** : son type qui peut être l'un des types suivants : 'alimentation', 'nettoyage', 'vêtement', 'appareil'.
- **stock** : La quantité en stock
- une constante **TVA** définit ainsi :

```
const TVA = [  
    '0.05' => ['alimentation', 'nettoyage'],  
    '0.2' => ['vetement', 'appareil']  
];
```

1. Créer cette classe ;
2. Ajouter à ces attributs des accesseurs et mutateurs, tel que :
 - a. La **référence** doit commencer par au moins deux lettres en majuscules ;
 - b. Le **prix** et le **stock** ne doivent pas être négatifs ;
 - c. Le type ne doit accepter que les types prédéfinis : 'alimentation', 'nettoyage', 'vêtement' et 'appareil'.
 - d. Le **type** doit être retourné en majuscule ;Levez des exceptions en cas d'erreurs.
3. Ajouter à cette classe les fonctions suivantes :
 - a. **valeurEnStock** : qui calcule et retourne la valeur du produit en stock (prix x stock)
 - b. **calculerTTC** : qui permet de chercher le taux TVA selon le type de produit, calculer et retourner son prix en tout taxe compris ;

- c. **solde (\$pourcentage)** : qui permet de retourner le nouveau prix après le solde.
- 4. Sous un dossier « tests », ajouter une classe ProduitTest, qui permettra de tester toutes les fonctions écrites en 2. et 3.

TP 5.1

Objectifs :

- Rappel des concepts de la POO
- Initiation aux tests unitaires

Exercice :

1. Définir une classe **Client** avec les attributs suivants : **CIN, Nom, Prénom, Tél.**
2. Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.
3. Définir un constructeur permettant d'initialiser tous les attributs.
4. Définir la méthode **Afficher ()** permettant d'afficher les informations du Client en cours.
5. Créer Une classe **Compte** caractérisée par son **solde** et un **code** qui est incrémenté lors de sa création ainsi que son **propriétaire** qui représente un client.
6. Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe (le numéro de compte est en lecture seule)
7. Définir un constructeur permettant de créer un compte en indiquant son propriétaire.
8. Ajouter à la classe **Compte** les méthodes suivantes :
 - i. Une méthode permettant de **Crediter()** le compte, prenant une somme en paramètre.
 - ii. Modifier la méthode **Crediter()** afin de pouvoir prendre en arguments, en plus de la somme, un compte bancaire, et de créditer le compte et de débiter le compte passé en paramètres.
 - iii. Une méthode permettant de **Debiter()** le compte, prenant une somme en paramètre
 - iv. Modifier la méthode **Debiter ()** afin de pouvoir prendre en arguments, en plus de la somme, un compte bancaire, et de débiter le compte et de créditer le compte passé en paramètres.
 - v. Une méthode qui permet d'afficher le résumé d'un compte.
 - vi. Une méthode qui permet d'afficher le nombre des comptes créés.
9. A l'aide de PHPUnit, testez les différentes méthodes créées.

TP 5.2

Objectifs :

- Rappel des concepts de la POO
- Initiation aux tests unitaires

Soit la classe `salarie.php` dont le code est fourni à la fin de ce document ;

La classe `salarie` est caractérisé par les attributs suivants :

- `matricule`;
 - `nomComplet`;
 - `salaire`;
 - `static tauxCS = 20`;
 - `$dateEmbauche`; (sous forme de m/d/y)
1. Ajouter une méthode « `primeAnnuelle` » permettant de calculer la prime de fin d'année en se basant sur le salaire et le nombre d'années d'expériences, en suivant la formule ci-après :
$$\text{Salaire} \times 0.8 + 100 \times \text{nombre d'années d'expériences}$$
 2. Ecrire le code permettant de tester les méthodes :
 - a) `setMatricule`
 - b) `getDateEmbauche`
 - c) `experience`
 - d) `calculerSalaireNet`
 - e) `primeAnnuelle`
 3. Analyse de la qualité du code et de la couverture du code :
 - Générer la configuration de `phpunit` ;
 - A l'aide de `phpunit` et `xdebug`, générez les rapports d'analyse des tests ;
 - Créer le fichier « `sonar-project.properties` » dans la racine du projet et configurer les différentes paramètres concernant la clé du projet ainsi que les rapports d'analyse ;
 - Créer un nouveau projet sous Sonarqube (avec la même clé du projet fournie) ;
 - Récupérer la commande `sonar-scanner` permettant de lancer l'analyse ;

- Lancer le scan en exécutant la commande ;
- Consulter les résultats sous Sonarqube.

La classe « salarié » :

```
<?php
use DateTime;

class salarié
{
    private int $matricule;
    private string $nomComplet;
    private float $salaire;
    public static float $tauxCS = 20;
    private DateTime $dateEmbauche;

    public function __construct(int $matricule = 0, string $nomComplet = "",
float $salaire = 0, $dateEmbauche = "")
    {
        $this->matricule = $matricule;
        $this->nomComplet = $nomComplet;
        $this->salaire = $salaire;
        // $dateEmbauche est une chaîne sous forme de m/d/y
        $this->dateEmbauche = $dateEmbauche != "" ? new
DateTime($dateEmbauche) : new DateTime("now");
    }
    //
    public function setMatricule(int $matricule): void
    {
        if (!preg_match("/^\d{3,7}$/", $matricule))
            throw new Exception("Matricule invalide! ");
        $this->matricule = $matricule;
    }

    public function getMatricule(): int
    {
        return $this->matricule;
    }

    ///
    public function __toString()
    {
        return "Salarié : Matricule: $this->matricule, Nom complet: $this-
>nomComplet, Salaire: $this->salaire <br>";
    }
}
```

```

public function experience()
{
    $today = new DateTime("now");
    $difference = $this->dateEmbauche->diff($today);
    return $difference->format("%y");
}
//
public function calculerSalaireNet(): float
{
    return $this->salaire - ($this->salaire * self::$tauxCS / 100);
}

/**
 * Get the value of dateEmbauche
 */
public function getDateEmbauche()
{
    return $this->dateEmbauche;
}
}

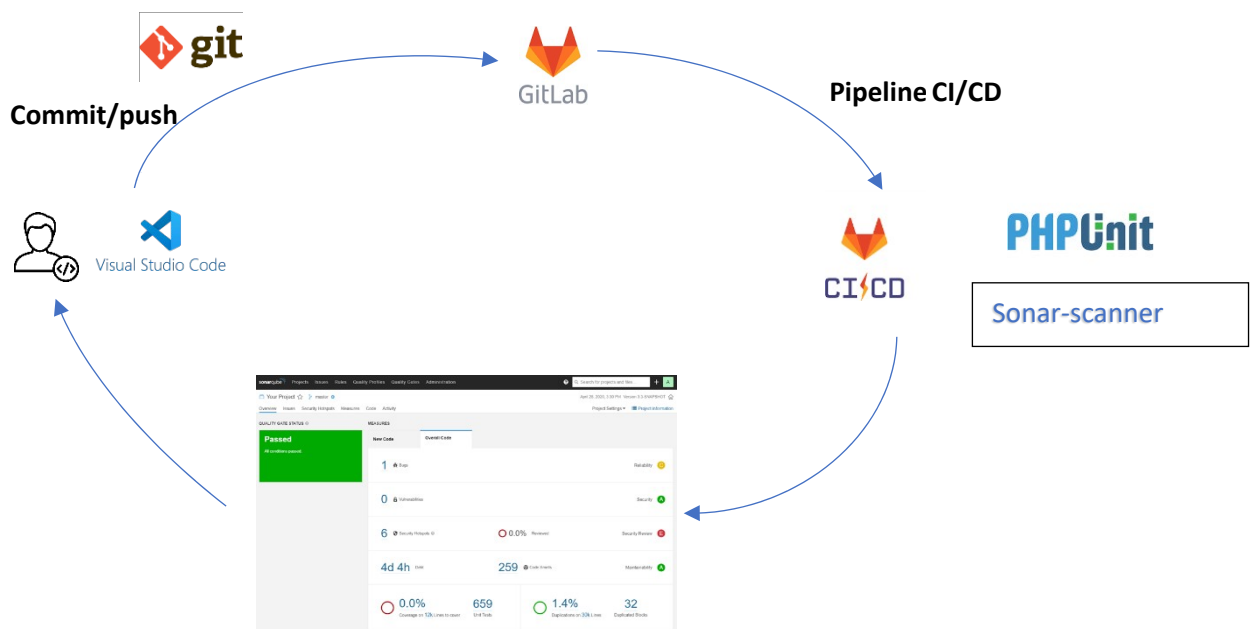
```

TP 6

Objectifs :

- Manipuler un pipeline CI/CD sous Gitlab

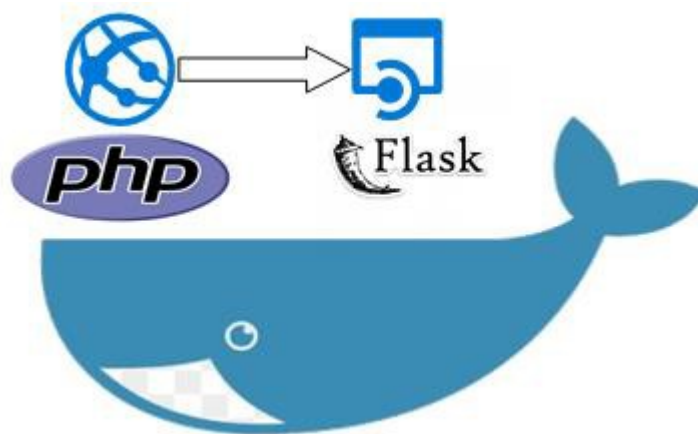
On souhaite configurer un pipeline automatisant le lancement des tests unitaires ainsi que l'analyse de la qualité de code avec sonar-scanner ; l'illustration ci-après montre l'architecture à mettre en place :



1. Choisir un projet de votre choix ;
2. Configurer les tests unitaires nécessaires (en utilisant phpunit) ;
3. Créer un projet sous Sonarqube ;
4. Récupérer la commande sonar-scanner et configurer le pipeline permettant d'automatiser ses processus ;
5. A chaque commit/push, le pipeline démarrera pour exécuter les différentes tests et analyse ;

Liste des étudiants

Ce dépôt est une application simple pour lister les étudiants avec un serveur Web (PHP) et une API (Flask)



Objectifs

Les objectifs de ce devoir pratique sont de garantir que vous êtes capable de gérer une infrastructure Docker, vous serez donc évalué sur les points suivants

Thèmes:

- améliorer un processus de déploiement d'application existant
- gestion des versions de votre infrastructure
- aborder les meilleures pratiques lors de la mise en œuvre de l'infrastructure Docker
- Infrastructure en tant que code

Contexte

CCNTechnologies est une société informatique située au Cameroun et développe des logiciels pour les lycées.

Le département de l'innovation souhaite perturber l'infrastructure existante pour garantir qu'il peut être évolutif, facilement déployé avec un maximum d'automatisation.

CCNTechnologies souhaite que vous construisiez un « **POC** » pour montrer comment Docker peut vous aider et à quel point cette technologie est efficace.

Pour ce POC, *CCNTechnologies* vous donnera une application et vous demandera de construire une infrastructure « découplée » basée sur « **Docker** ».

Actuellement, l'application fonctionne sur un serveur unique avec aucune évolutivité et aucune haute disponibilité.

Lorsque *CCNTechnologies* doit déployer une nouvelle version, il y a toujours un problème.

En conclusion, *CCNTechnologie* a besoin d'agilité dans sa ferme logicielle.

Infrastructure

Pour ce POC, vous n'utiliserez qu'une seule machine avec un docker installé dessus.

La construction et le déploiement seront effectués sur cette machine.

CCNTechnologies vous recommande d'utiliser le système d'exploitation centos7.6 car c'est le plus utilisé dans l'entreprise.

Veuillez également noter que vous êtes autorisé à utiliser une machine virtuelle basée sur Centos7.6 et non votre machine physique.

La sécurité est un aspect très critique de *CCNTechnologies* DSI, veuillez donc ne pas désactiver le pare-feu ou d'autres mécanismes de sécurité, sinon veuillez expliquer vos raisons dans votre livraison.

Application

L'application sur laquelle vous allez travailler s'appelle " *student_list* ", cette application est très basique et permet à *CCNTechnologies* d'afficher la liste des étudiants avec leur âge.

student_list comporte deux modules :

- le premier module est une API REST (avec authentification de base nécessaire) qui envoie la liste de souhaits de l'étudiant en fonction du fichier JSON

- Le deuxième module est une application Web écrite en HTML + PHP qui permet à l'utilisateur final d'obtenir une liste d'étudiants

Votre travail consiste à construire un conteneur pour chaque module et à les faire interagir les uns avec les autres

Les fichiers que vous devez fournir (dans votre livraison) sont **Dockerfile** et **docker-compose.yml** (actuellement les deux sont vides)

Il est maintenant temps de vous expliquer le rôle de chaque fichier :

- docker-compose.yml : pour lancer l'application (API et application web)
- Dockerfile : le fichier qui sera utilisé pour construire l'image de l'API (les détails seront donnés)
- requirements.txt : contient tous les packages à installer pour exécuter l'application
- student_age.json : contient le nom de l'étudiant avec son âge au format JSON
- student_age.py : contient le code source de l'API en python
- index.php : Page PHP à laquelle l'utilisateur final se connectera pour interagir avec le service et répertorier les élèves par âge. Vous devez mettre à jour la ligne suivante avant d'exécuter le conteneur du site web afin d'adapter **api_ip_or_name** et **le port** à votre déploiement.

```
$url = 'http://<api_ip_or_name:port>/ccntechnologies/api/v1.0/get_student_ages';
```

Construire et tester (7 points)

CCNTECNOLOGIES vous donnera des informations pour construire le conteneur API

- Image de base

Pour créer une image API, vous devez utiliser « python:3.11-slim »

- Mainteneur

N'oubliez pas de préciser les informations du mainteneur

- Ajouter le code source

Vous devez copier le code source de l'API dans le conteneur à la racine du chemin "/"

- Condition préalable

L'API utilise le moteur FLASK, vous devez installer un package

```
apt update -y && apt install python-dev python3-dev libsasl2-dev python-dev  
libldap2-dev libssl-dev -y
```

Copiez le fichier requirements.txt dans le conteneur du répertoire racine « / » pour installer les packages nécessaires au démarrage de notre application

pour lancer l'installation, utilisez cette commande

```
pip3 install -r /requirements.txt
```

- Données persistantes (volume)

Créez un dossier de données à la racine « / » où les données seront stockées et déclarez-le comme un volume

Vous utiliserez ce dossier pour monter la liste des étudiants

- Port API

Pour interagir avec cette API, exposez le port 5000

- CMD

Lorsque le conteneur démarre, il doit exécuter le student_age.py (copié à l'étape 4), il devrait donc ressembler à quelque chose comme

```
CMD [ "python3", "./student_age.py" ]
```

Créez votre image et essayez de l'exécuter (n'oubliez pas de monter le fichier *student_age.json* dans */data/student_age.json* dans le conteneur), vérifiez les journaux et vérifiez que le conteneur écoute et est prêt à répondre

Exécutez cette commande pour vous assurer que l'API répond correctement (prenez une capture d'écran à des fins de livraison) NB : démarrez votre conteneur en utilisant ce port spécifique pour l'atteindre Port : 5000

```
curl -u toto:python -X GET http://<host IP>:<API exposed  
port>/ccntechnologies/api/v1.0/get_student_ages
```

Félicitations ! Vous êtes maintenant prêt pour l'étape suivante (docker-compose.yml).

Infrastructure en tant que code (5 points)

Après avoir testé votre image API, vous devez tout assembler et le déployer à l'aide de docker-compose.

Le fichier **docker-compose.yml** va déployer deux services :

- site Web : l'interface utilisateur finale avec les caractéristiques suivantes
 - image : php:apache - environnement : vous fournirez le NOM D'UTILISATEUR et le MOT DE PASSE pour permettre à l'application Web d'accéder à l'API via l'authentification
 - Volumes : pour éviter que l'image php:apache ne soit exécutée avec le site web par défaut, nous allons lier le site web fourni par CCNTEchnologies. Vous devez avoir un fichier similaire. `./website:/var/www/html`
 - dépend de : vous devez vous assurer que l'API démarrera avant le site Web
 - port : n'oubliez pas d'exposer le port
- API : l'image créée précédemment doit être utilisée avec la spécification suivante
 - image : le nom de l'image construite précédemment - volumes : Vous monterez le fichier `student_age.json` dans `/data/student_age.json`
 - port : n'oubliez pas d'exposer le port
 - réseaux : n'oubliez pas d'ajouter un réseau spécifique pour votre projet

Supprimez votre conteneur précédemment créé

Exécutez votre `docker-compose.yml`

Enfin, accédez à votre site Web et cliquez sur le bouton « Lister les étudiants »

Si la liste des étudiants apparaît, l'intégration de l'application CCNTECNOLOGIES est réussie ! Félicitations ! (Faites une capture d'écran)

Registre Docker (4 points)

CCNTECNOLOGIES a besoin que vous déployiez un registre privé et stockiez les images créées

Vous devez donc déployer :

- [un registre](#) Docker
- [une interface](#) Web pour voir l'image poussée comme un conteneur

Ou vous pouvez utiliser [Portus](#) pour exécuter les deux

N'oubliez pas de pousser votre image sur votre registre privé et de la montrer dans votre livraison.

Livraison (4 points)

Votre livraison doit être un lien de votre dépôt avec votre nom qui contient :

- Un fichier README avec vos captures d'écran et explications.
- Fichiers de configuration utilisés pour réaliser l'exercice noté (docker-compose.yml et Dockerfile).

Votre livraison sera évaluée sur :

- Qualité des explications
- Qualité des captures d'écran (pertinence, visibilité)
- Qualité de la présentation
- La structure de votre dépôt github